

# Runbook de Operacao, Troubleshooting e Manutencao — Addon FastChannel

---

Guia operacional para diagnostico, suporte e manutencao em producao.

Gerada em 2026-07-01. Ver tambem DEVELOPER\_GUIDE.md (arquitetura/build) e CHANGELOG.md (historico).

---

# OPERAÇÃO, TROUBLESHOOTING & MANUTENÇÃO (Runbook)

Esta seção é o guia operacional do addon Sankhya↔FastChannel para quem vai manter, dar suporte e diagnosticar em produção. Tudo aqui foi conferido contra o código real da branch `merge-unify` ( `model/src/main/java/br/com/bellube/fastchannel/` ), o `CHANGELOG.md` (v1.2.x) e os scripts em `X:\tmp_sql`.

## 1. Os jobs de sync e o que observar

Os jobs são agendados pelo **fallback interno** em `FastchannelAutoProvisioning.startInternalFallback()` ( `installation/FastchannelAutoProvisioning.java:235-282` ), que sobe um `ScheduledExecutorService` de **10 threads** (linha 253; pool foi de 5→10 para evitar starvation dos jobs pesados). As threads têm nome `fastchannel-auto-<id>` (linha 242) — útil para filtrar no log. Se houver Ações Agendadas nativas do módulo, o scheduler nativo assume no lugar do fallback ( `tryStartNativeScheduledActions` , linha 168).

Todos os jobs só rodam se `FastchannelConfig.isAtivo()` for verdadeiro (checado dentro de cada wrapper `schedule` / `scheduleDynamic` e no início de cada `executeScheduler` ).

Tabela dos jobs agendados

| Job (task name)   | Classe                                                  | Intervalo padrão                                               | Como é agendado                                                                                       | Property/config de override                     |
|-------------------|---------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| outbox            | <code>OutboxProcessorJob</code>                         | <code>INTERVAL_QUEUE</code> min (default <b>2 min</b> , min 1) | <code>scheduleDynamic</code> , initialDelay 60s ( <code>FastchannelAutoProvisioning.java:261</code> ) | coluna <code>AD_FCCONFIG.INTERVAL_QUEUE</code>  |
| order-import      | <code>OrderImportJob</code>                             | <code>INTERVAL_ORDERS</code> min (default 5 min)               | <code>scheduleDynamic</code> , initialDelay 30s ( :258 )                                              | coluna <code>AD_FCCONFIG.INTERVAL_ORDERS</code> |
| auto-sweep-prices | <code>AutoPriceChangesSweepJob</code>                   | <b>10 min</b>                                                  | <code>schedule</code> ( :277 )                                                                        | <code>-Dfc.auto.sweep.minutes</code>            |
| price-full        | <code>PriceFullSyncJob</code>                           | <b>6 h</b>                                                     | <code>schedule</code> , initialDelay 120s ( :269 )                                                    | <code>-Dfc.auto.price.hours</code>              |
| stock-full        | <code>StockFullSyncJob</code>                           | <b>6 h</b>                                                     | <code>schedule</code> , initialDelay 120s ( :271 )                                                    | <code>-Dfc.auto.stock.hours</code>              |
| status-sync       | <code>OrderStatusSyncJob</code>                         | 3 min                                                          | <code>schedule</code> ( :264 )                                                                        | <code>-Dfc.auto.status.minutes</code>           |
| depara-sync       | <code>DeparaService.syncProductDeparaFromRefforn</code> | 24 h                                                           | <code>schedule</code> ( :256 )                                                                        | <code>-Dfc.auto.depara.hours</code>             |

**Nota crítica de wiring (v1.2.80):** `schedule()` ( :338-371 ) converte **tudo para SECONDS** antes de `scheduleWithFixedDelay` . O bug histórico (initialDelay em segundos passado com `unit=HOURS` = 5 dias) fazia `price-full` / `stock-full` **nunca rodarem**. Se você mexer nesse método, valide o log `AutoProvisionamento[<nome>]: agendado initialDelay=<n>s, periodo=<n>s.` ( :369 ) — os valores devem ser em segundos coerentes (ex.: `stock-full` → `periodo=21600s` ).

### 1.1 OutboxProcessorJob — o coração do sync

- Ponto de entrada: `OutboxProcessorJob.executeScheduler()` ( `job/OutboxProcessorJob.java:74` ).
- Processa a fila `AD_FCQUEUE` em lotes de `AD_FCCONFIG.batchSize` (default 50, `FastchannelConstants.DEFAULT_BATCH_SIZE` ).
- Antes de buscar pendentes chama `reactivateErrorItems(DEFAULT_MAX_RETRIES=3)` ( :93 ) — reabilita itens em **ERRO** que ainda têm retry.
- **Claim atômico** por item: `tryMarkAsProcessing(idQueue)` ( :114 ) evita processamento duplo se dois ciclos rodarem em paralelo.
- Roteamento por `ENTITY_TYPE` (switch em :120): `ESTOQUE` , `PRECO` , `PRODUTO` , `TRACKING` , `PEDIDO_STATUS` . Tipo desconhecido → `markAsFatalError(..., "Tipo desconhecido")` .
- **O que observar no log:** `=== Iniciando Job de Processamento Outbox Fastchannel === ... Job concluido. Processados: X, Erros: Y.`
- **Terminadores de retry** (marcam `SUCCESS` para parar loop infinito, não são erro real): `isNonPublishableSkuError` (SKU não publicado, :571), `isFcStateTransitionError` (FC já adiantado, HTTP 400 `PossibleNextStatuses` , :591), `isFcInvoiceNotAcceptable` (NF já anexada, HTTP 404, :623), `isConfigurationError` ( :559 ) → **ERRO\_FATAL** imediato + e-mail via `NotificationService.notifyQueueFatalError` .

### 1.2 AutoPriceChangesSweepJob — rede de segurança de preços

- `AutoPriceChangesSweepJob.run()` ( `job/AutoPriceChangesSweepJob.java:97` ). Roda a cada 10 min.
- **Cenário 1 — promo escalonada expirada:** SQL `SQL_RECENTLY_EXPIRED_PROMOS` ( :78 ) busca `TGFDES.DTFINAL` no passado dentro da janela de **30 dias** ( `DEFAULT_EXPIRED_WINDOW_DAYS=63` ; era 2 dias antes da v1.2.91), com faixas em `TGFDQP` e versão mais recente por `NUPROMOCAO` . Cada promo é enfileirada **uma vez por vida do addon** via dedup `HANDLED_EXPIRED_PROMOS` ( :90 , set que zera no restart e re-varre os 30 dias).
- **Cenário 2 — TGFEFC alterado:** SQL `SQL_TGFEFC_CHANGED_SINCE` ( :94 ) pega `TGFEFC.CODPROD` com `DHALTREG > LAST_SWEEP` . O cursor `LAST_SWEEP` ( :49 ) é in-memory e no restart volta 1h atrás.
- Para cada produto: resolve SKU via `DeparaService.getSkuForStock` e `QueueService.enqueuePrice(codProd, sku)` ( :129 ) → vira `PRECO/UPDATE` no `AD_FCQUEUE` (com debounce de 5s do `QueueService` ).

- **Log a observar (por tick):** `[AutoSweep] varredura ok: expired=N tgfecc=M enfileirados=K sem_sku=X erro=Y`. Se `expired=0` persistente enquanto operação reclama de escalonado preso, verifique se a promo está fora da janela de 30 dias ou se o addon ficou fora do ar > 30 dias.

### 1.3 PriceFullSyncJob — full sync 6h com skip de inexistentes na FC

- `PriceFullSyncJob.executeScheduler()` ( `job/PriceFullSyncJob.java:27` ): monta lista de `CODPROD` via `FastchannelProductFilter.getActiveFcProductsSql` (marcas `AD_FAST='S'`) e chama `PriceService.syncPriceBatch(codProds)`.
- `syncPriceBatch` ( `service/PriceService.java:173` ) roda **paralelo com 4 workers** ( `-Dfc.sync.parallelism`, threads `fc-sync-price-<id>` ).
- **Otimização v1.2.89 (skip de inexistentes):** prefetch `getFcExistingSkus()` ( `:278` ) agrega os SKUs que realmente existem na FC (via `listExistingSkus`, cache 5 min). Produtos cujo SKU não está no conjunto são **pulados** ( `shouldSkipNonexistentSku`, ~75% do catálogo `AD_FAST` não existe na FC e só geraria HTTP 404). **Fail-open:** se prefetch vier vazio/falhar retorna `null` → nada é pulado (comportamento legado). Desliga com `-Dfc.sync.skipNonexistent=false`.
- **Log final a observar:** `[syncPriceBatch] concluido: N produtos em Xs (A ok, B falhas, C pulados por não existir na FC)` e `[SYNC-OPT] prefetch FC concluido: N SKU(s) existentes (cache 5min)`.

### 1.4 StockFullSyncJob — full sync de estoque 6h

- `StockFullSyncJob.executeScheduler()` ( `job/StockFullSyncJob.java:27` ).
- Query ( `:56` ) itera `TGFEST` para produtos FC ativos, **filtrando por** `config.getCodEmp()` e `config.getCodLocal()` (v1.2.75 — sem esse filtro, o último PUT por empresa sobrescrevia o estoque no FC, vendendo sem ter). Produto inativo ( `TGFPRO.ATIVO<>'S'` ) é enviado com qty **0** ( `:107` ).
- **Log a observar:** `[StockFullSyncJob] Filtros aplicados: codEmp=26 codLocal=TODOs e StockFullSyncJob concluido. Enviados: X, Erros: Y`.
- Muitos "Erros" no full sync de estoque costumam ser SKU inexistente na FC (não há skip aqui como no de preço) — não necessariamente incidente.

## 2. PrecoListener — dispara na alteração de preço (fix da instância)

- `listener/PrecoListener.java:32` — `@Listener(instanceNames = {"Excecao", "ExcecaoPreco"})`.
- **Regra de ouro (v1.2.90):** a instância correta do `TGFEXC` no dicionário Sankhya é **Excecao** (não `ExcecaoPreco`, que não existe). Antes do fix o listener **nunca disparava** na tela de Gestão de Preços; as alterações só chegavam pelo `AutoPriceChangesSweepJob` (lag de até ~15 min). `ExcecaoPreco` foi mantido no array por segurança (instância inexistente apenas nunca dispara).
- Fluxo: `afterInsert/afterUpdate/afterDelete` → `processPrecoChange` ( `:52` ) → filtra por tabelas elegíveis ( `PriceTableResolver.resolveEligibleTables` ), resolve SKU e `QueueService.enqueuePrice`. Log de sucesso: `Preco enfileirado: CODPROD X (SKU Y)`.
- **Diagnóstico rápido:** se uma alteração de preço na tela **não** aparece na FC em ~1-2 min, filtre o log por `Preco enfileirado` numa thread `default task-*` (é o listener reagindo à UI). Se só aparecer em thread `fastchannel-auto-*`, quem pegou foi o sweep (listener não disparou) — sinal de regressão do nome da instância.

## 3. Índice de Incidentes (v1.2.85 → v1.2.91) — guia de troubleshooting

Tabela derivada do `CHANGELOG.md`. Use como primeira parada quando um sintoma reaparecer.

| Versão                    | Sintoma reportado                                                                                                                      | Causa raiz                                                                                                                                                                                                                                                                                            | Fix                                                                                                                                                                                                                                                  |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>v1.2.85</b><br>(25/05) | Preço escalonado <b>duplicado</b> no site e nunca limpo (impressão de "todas a partir de 1"); faixa 1-3 do SKU 31251453 postada 6x     | <code>FastchannelPriceClient.listPriceBatches()</code> lia campo JSON <code>ProductPriceBatch</code> (nome do <b>elemento XML</b> ), mas a FC responde batches em <code>Payload[]</code> → sempre lista vazia → dedup e limpeza cegos (0 deletes, re-POST a cada sync; FC <b>não deduplica POST</b> ) | <code>listPriceBatches</code> lê <code>Payload</code> ; <code>updatePriceBatches</code> faz self-heal (mantém 1 de cada faixa, deleta duplicatas/obsoletas); <code>PriceBatchResolver</code> faixa-topo vira "N e acima" ( <code>Max=999999</code> ) |
| <b>v1.2.86</b><br>(26/05) | Faixas pararam de duplicar, mas Qtd Mín/Máx gravava <b>0</b> (tudo "a partir de 1 unid.")                                              | FC grava <code>Min/MaxBatchSize=0</code> quando recebe número em <b>notação decimal</b> ( <code>"1.0"</code> ); <code>TGFDPQ.QTDE</code> é <code>float</code> → gson emitia <code>"1.0"</code>                                                                                                        | <code>PriceBatchResolver</code> normaliza escala 0 ( <code>toIntScale</code> ); <code>FastchannelPriceClient.normalizeDesiredBatches</code> faz <code>setScale(0)</code> no chokepoint único de todo POST → FC grava 1..3, 4..5, 6..999999           |
| <b>v1.2.87</b><br>(26/05) | Listagem/carrinho mostravam Preço de Venda cheio enquanto detalhe mostrava escalonado                                                  | Regra de negócio: faixa "a partir de 1 un." deve sobrepor o Preço de Venda (campanha com prioridade)                                                                                                                                                                                                  | <code>PriceService.syncPriceTable</code> resolve batches <b>antes</b> do PUT; se faixa cobre a 1ª unidade, <code>SalePrice</code> é sobreposto ( <code>findFirstUnitBatchPrice</code> ); <code>ListPrice</code> mantido como "De" riscado            |
| <b>v1.2.88</b><br>(01/06) | Escalonado expirado não limpo automaticamente; alteração manual em TGFEXC nem sempre disparava o listener                              | Sem evento JAPE quando <code>TGFDES.DTFINAL</code> passa; edição de TGFEXC fora do JAPE não enfileira                                                                                                                                                                                                 | Novo <code>AutoPriceChangesSweepJob</code> a cada 10 min: enfileira PRECO/UPDATE p/ promos expiradas e TGFEXC alterados                                                                                                                              |
| <b>v1.2.89</b><br>(01/06) | Full sync lento e painel cheio de "erros"                                                                                              | ~2544 produtos <code>AD_FAST</code> mas só ~611 existem na FC → ~1900 HTTP 404 por sync (× 12 tabelas). Addon nunca cria produto no catálogo FC                                                                                                                                                       | <code>getFcExistingSkus()</code> (prefetch, cache 5 min, <b>fail-open</b> ) faz <code>syncPriceBatch</code> / <code>syncAll</code> pularem SKU inexistente. <code>-Dfc.sync.skipNonexistent=false</code> desliga                                     |
| <b>v1.2.90</b><br>(01/06) | Alteração de preço na tela "não alterava" no site; só chegava ~16-45 min depois                                                        | <code>PrecoListener</code> registrado para instância <b>ExcecaoPreco</b> (inexistente) → nunca disparava; só o sweep (poll 10 min) capturava                                                                                                                                                          | <code>@Listener(instanceNames={"Excecao","ExcecaoPreco"})</code> — reage à entidade real <code>Excecao</code> ; alteração agora vai ao outbox em ~1 min                                                                                              |
| <b>v1.2.91</b><br>(15/06) | Preço escalonado que deveria ter terminado continuava ativo (ex.: produto 9102/SKU 31013353, promo 418 fim 10/06 ainda ativa em 15/06) | Janela do <code>AutoPriceChangesSweepJob</code> era só <b>2 dias</b> ; se o addon/full sync ficasse fora do ar > 2 dias após a expiração, o escalonado só saía no próximo full sync completo (~4h)                                                                                                    | Janela 2 → <b>30 dias</b> ( <code>-Dfc.auto.sweep.expiredDays</code> ) + dedup por <code>NUPROMOCAO</code> ( <code>HANDLED_EXPIRED_PROMOS</code> ) para não re-enfileirar a cada tick                                                                |

**Padrão dos incidentes de preço:** quase todos nascem do par **PriceBatchResolver (monta faixas) + FastchannelPriceClient (lê/posta batches) + sweep/listener (dispara)**. Ao investigar preço/escalonado, comece por esses três.

## 4. Acesso a PROD para diagnóstico

### 4.1 Banco (SQL Server) — via PowerShell .NET SqlClient

`sqlcmd` **não está instalado** no ambiente. Use o helper `X:\tmp_sql\q.ps1` (`.NET System.Data.SqlClient`):

```
powershell -File X:\tmp_sql\q.ps1 -Sql "SELECT TOP 20 IDQUEUE, ENTITY_TYPE, STATUS, RETRY_COUNT, ENTITY_KEY, ERROR_MSG FROM AD_FCQUEUE WHERE STATUS IN ('ERRO','ERRO_FATAL','PROCESSANDO') ORDER BY IDQUEUE DESC"
```

Connection string embutida no `q.ps1` (linha 2):

```
Server=bellube-sql01-oci.cldns.top,1433;Database=sankhya_prod;User Id=sankhya;Password=azsxdc;TrustServerCertificate=True;Connect Timeout=20
```

Tabelas de diagnóstico mais usadas: `AD_FCQUEUE` (fila/outbox), `AD_FCLOG` / `AD_FCLOGS` (logs de operação), `AD_FCPEDIDO` (pedidos importados), `AD_FCDEPARA` (De-Para SKU→CODPROD, tabelas de preço, storage/reseller), `AD_FCONFIG` (config: `INTERVAL_QUEUE`, `INTERVAL_ORDERS`, `CODEMP`, `CODLOCAL`, chaves por canal). No lado Sankhya: `TGFCAB` / `TGFITE` (`AD_NUMFAST` marca pedido FC), `TGFEXC` (preço), `TGFDES` / `TGFDPQ` (promo escalonada), `TGFEST` (estoque), `TGFMAR` (`AD_FAST` / `AD_FASTREF`).

### 4.2 FC API — OAuth (client\_credentials Azure AD)

Endpoint de token (mesmo do addon, `FastchannelConstants.AUTH_URL` e `FastchannelTokenManager`):

`https://login.microsoftonline.com/fastchannel.com/oauth2/v2.0/token`, `grant_type=client_credentials`. Bases (`FastchannelConstants`): `Orders` `.../order-management/v1`, `Stock` `.../stock-management/v1`, `Price` `.../price-management/v1`. Além do Bearer, as chamadas precisam da **subscription key** por canal nos headers `Ocp-Apim-Subscription-Key` / `Subscription-Key`. Os scripts do \$5 já trazem `client_id/secret/scope` e a `subscription key` de consumo (`kc`) prontos para uso.

## 5. Scripts de reconciliação / force-sync (FC vs Sankhya)

Ficam em `X:\tmp_sql`. Mapa fixo tabela FC → NUTAB vigente usado por todos: `3=4460, 4=4459, 24=4462, 25=4461, 26=4458, 27=4457`.

- `X:\tmp_sql\recon_all.ps1` — reconciliação: puxa `SNK_GET_PRECO` das 6 NUTABs por SKU (via `AD_FCDEPARA`) do BD, puxa os preços das 6 `PriceTableId` da FC (paginado, `Payload[].SalePrice`) e conta **MISMATCH** por tabela. Saída final: `=== TOTAL MISMATCH (todas tabelas): N ===`. Rode primeiro para medir a divergência.
- `X:\tmp_sql\force_push_all.ps1` — force-sync: re-empurra (PUT `/prices/{sku}`) `SalePrice + ListPrice` das 6 tabelas para todo SKU que existe na FC. Saída: `tabela T (NUTAB X): A ok, B erros, C pulados` e `=== FORCE PUT COMPLETO: ... ===`. Use quando a reconciliação acusar mismatch e você quiser corrigir imediatamente sem esperar o full sync de 6h.
- Variantes: `recon_precos.ps1`, `force_push_t3.ps1` (só tabela 3), `fc.ps1` (helper de token/consulta pontual).

Ambos filtram `COD_EXTERNO NOT LIKE 'V%'` e `NOT LIKE 'M%'` e ignoram SKU sem preço válido (`sale <= 0`). Preços são comparados em centavos (`ROUND(...*100,0)`).

**Escopo:** estes scripts cobrem **preço** nas 6 tabelas. Para estoque/pedidos, use consultas diretas via `q.ps1` + o full sync do addon (`StockFullSyncJob`).

## 6. Gotchas operacionais conhecidos (não são bug do addon)

- **Bloqueio de crédito do parceiro** — `TGFPAR.BLOQUEAR='S'` faz o Sankhya **barrar pedido a prazo** desse parceiro. É comportamento nativo do ERP, **não é bug do addon**. Se um pedido FC não gera nota/fica preso por bloqueio de crédito, verifique `TGFPAR.BLOQUEAR` do CODPARC — a solução é liberar o crédito no cadastro/financeiro, não mexer no addon.
- **Cache da vitrine FastChannel** — a API pode estar **correta** (confira via `recon_all.ps1` /GET direto) enquanto o **site** ainda mostra preço/estoque antigo por cache da vitrine. Antes de tratar como incidente, confirme na API; se a API bate com o Sankhya, é cache da FC (aguardar ou acionar o suporte FC). Esse foi exatamente o mal-entendido do \$v1.2.90 (operadora via cache + lag).
- **Disco X: enchendo com logs extraídos** — cada análise de `server.log.zip` gera pastas em `log_extracted/` e `logs/`, além de `hs_err_pid*.log` / `replay_pid*.log` na raiz do worktree. Limpe periodicamente (`log_extracted/`, `*.zip` de log antigos, `hs_err_pid*`, `replay_pid*`) para não estourar o X:.

## 7. Como analisar um server.log (zip)

Fluxo padrão de triagem de um `server.log_YYYYMMDDHHMMSS.zip` do WildFly de PROD:

1. **Extrair** o zip para `log_extracted/` (ou `logs/`).
2. **Filtrar por marcador** conforme o subsistema investigado:
  - `[AutoSweep]` → varredura de preços (expired/tgfcx/enfileirados).
  - `[syncPriceBatch]` e `[SYNC-OPT]` → full sync de preço e prefetch de SKUs existentes.
  - `[StockFullSyncJob]` → filtros e resultado do full sync de estoque.
  - `PriceBatchResolver` → montagem de faixas escalonadas (linhas `PriceBatchResolver: CODPROD=... total rows=... items=...`, `faixa topo`, `promocao expirada`).
  - `[ServiceInvoker]` / `[InternalAPI]` → criação de pedido (estratégia usada e NUNOTA).
  - `[VLRNOTA-REPAIR]` → correção de total de nota.
  - `AutoProvisionamento[<job>]` → confirmação de agendamento (initialDelay/período em segundos).
3. **Filtrar por pedido:** grep pelo `orderId` FC (ex.: `4779`) e pelo `NUNOTA` correspondente para seguir o pedido ponta-a-ponta (import → status → NF).
4. **Filtrar por classe/thread:** grep pelo FQCN do job (`OutboxProcessorJob`, `PrecoListener`) ou pelo nome da thread (`fastchannel-auto-*` = jobs do scheduler; `default task-*` = evento JAPE reagindo à UI; `fc-sync-price-*` = workers do full sync de preço).
5. **Erros:** procure `SEVERE`, `WARNING`, `ClassCastException`, `Tipo desconhecido`, `ERRO_FATAL`, e as mensagens dos detectores do outbox (`OrderStatusId`, `PossibleNextStatuses`, `Resource not found`) para classificar se é loop de retry legítimo ou falso positivo já tratado.

Encoding: em PROD os acentos costumam vir corrompidos (💎). Os detectores do `OutboxProcessorJob` já são invariantes a encoding (v1.2.82) — ao escrever novos filtros/detectores, use **palavras-chave sem acento**.